

本科论文"功能设计"部分 编写指南

以 Spring Boot 框架下的"课程信息查询功能"为例

第 4 章"主要功能模块设计"编写示范

本科论文“功能设计”部分编写指南

本指南以 Spring Boot 框架下的“课程信息查询功能”为例，示范本科论文第 4 章“主要功能模块设计”中一个典型小节应包含的内容要素及编写要点。学生可参照本指南的结构与思路，替换为自己的具体功能模块完成论文撰写。

1、三个类和接口完整的代码

以下给出 Web 后端实现“课程信息查询”功能所涉及 4 个核心文件（1 个 Controller、1 个 Service 接口、1 个 Service 实现类、1 个 Repository 接口）。为聚焦设计思路，功能中需要用到但非本示例重点的其他类（如统一响应封装类、课程实体类等），均以注释方式说明其用途，不再额外展开。

1.1 CourseController.java (Controller 层)

```
package com.example.demo.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;
import com.example.demo.common.Result;
import com.example.demo.entity.Course;
import com.example.demo.service.CourseService;

/**
 * 课程信息Controller类 (Web开放接口层)
 * 负责接收前端HTTP请求，调用Service层完成业务处理，返回统一JSON响应
 */
@RestController
@RequestMapping("/api/course")
public class CourseController {

    @Autowired
    private CourseService courseService;

    /**
     * 根据课程ID查询课程信息
     * @param id 课程ID (从URL路径中获取)
     * @return 统一响应结果，包含查询到的课程信息
     */
    @GetMapping("/{id}")
    public Result findById(@PathVariable Long id) {
        // 调用业务逻辑层查询课程信息
        Course course = courseService.findById(id);
        // 封装成功响应返回前端
        return Result.success(course);
    }
}
```

1.2 CourseService.java (Service 接口层)

```
package com.example.demo.service;

import com.example.demo.entity.Course;

/**
 * 课程信息业务逻辑服务接口
 * 定义课程模块对外暴露的公共业务能力
 */
public interface CourseService {

    /**
     * 根据课程ID查询课程信息
     * @param id 课程ID
     * @return 课程信息对象，若不存在则返回null
     */
    Course findById(Long id);
}
```

1.3 CourseServiceImpl.java (Service 实现层)

```
package com.example.demo.service.impl;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;
import com.example.demo.entity.Course;
import com.example.demo.repository.CourseRepository;
import com.example.demo.service.CourseService;

/**
 * 课程信息业务逻辑服务实现类
 * 实现CourseService接口，封装课程查询相关的核心业务逻辑
 */
@Service
public class CourseServiceImpl implements CourseService {

    @Autowired
    private CourseRepository courseRepository;

    @Override
    public Course findById(Long id) {
        // (1) 调用数据访问层，根据课程ID查询数据库
        // 返回查询到的课程实体对象，若不存在则返回null
        return courseRepository.findById(id).orElse(null);
    }
}
```

1.4 CourseRepository.java (Repository / DAO 层)

```
package com.example.demo.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import com.example.demo.entity.Course;

/**
 * 课程信息数据访问接口 (Repository / DAO层)
 * 继承JpaRepository, 由Spring Data JPA自动生成基本的增删改查实现
 */
@Repository
public interface CourseRepository extends JpaRepository<Course, Long> {

    /**
     * 根据课程编号查询课程信息
     * 方法命名符合Spring Data JPA规范, 框架会自动生成查询实现
     * @param courseCode 课程编号 (如: CS101)
     * @return 课程信息对象, 若不存在则返回null
     */
    Course findByCourseCode(String courseCode);
}
```

2、图 4-11 课程信息查询过程

编写说明：

泳道图（活动图）的作用是让读者对该功能在“用户—前端—Web 后端”三个主体之间的**整体协作过程**建立宏观认识。在论文中放置此图，可以直观展示一次完整业务操作所经历的阶段与数据流转路径，为后续细化到代码层面的类设计和流程设计奠定上下文基础。学生在绘制时应注意：每个泳道对应一个参与方，活动节点使用动词短语描述，箭线表示操作的先后次序。

3、课程信息查询过程的正文说明

课程信息查询过程说明如下：

（1）用户进入前端课程查询页面，输入需要查询的课程ID，点击【查询】按钮。

（2）前端收集用户输入的课程ID，将其作为URL路径参数，通过 HTTP GET 请求发送至 Web 后端的 `/api/course/{id}` 接口。

（3）Web 后端接收请求后，由 `CourseController` 提取路径参数 `id`，转发给 `CourseServiceImpl` 处理核心业务逻辑。`CourseServiceImpl` 调用 `CourseRepository` 根据课程ID查询数据库，将查询结果以 JSON 格式返回给前端，前端解析后向用户展示课程详细信息。

编写说明：

过程正文说明的目的是对泳道图中的每一步进行**文字级展开解释**，把“图里发生了什么”翻译成“系统具体做了什么、数据如何传递”。这部分文字起到承上启下的作用：向上对应泳道图的整体流程，向下为后文类设计和顺序图的细节提供铺垫。学生编写时应逐条对应图中的活动节点，使用“谁做了什么、产生了什么数据、发送给谁”的句式，避免笼统描述。

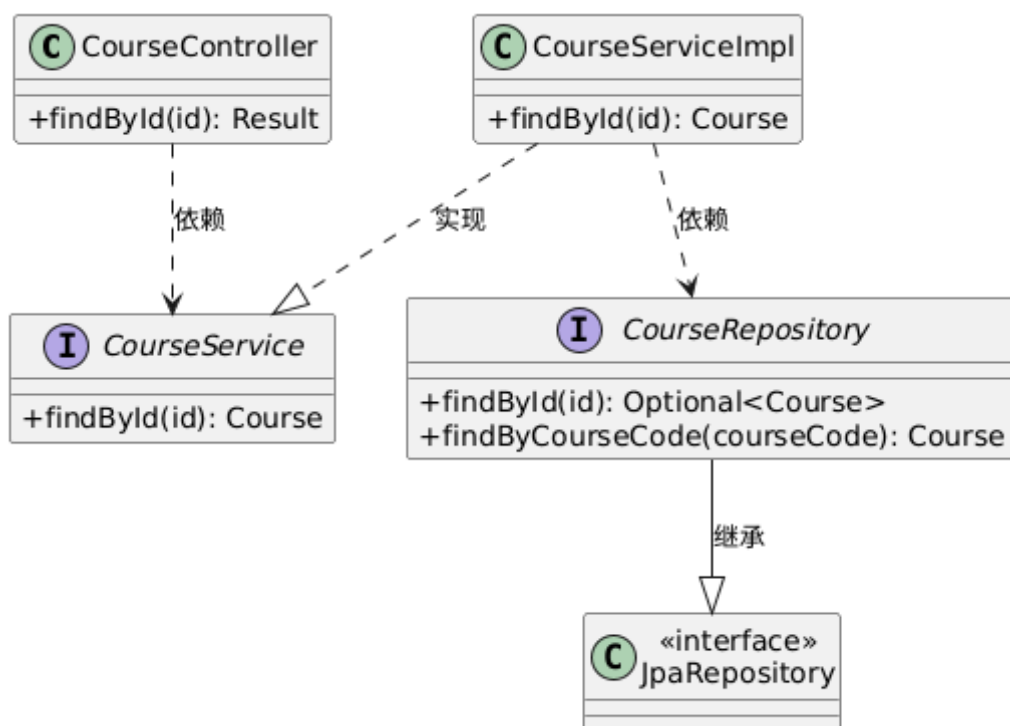
4、表 4-4 Web后端课程信息查询功能类

类名	描述
CourseController	课程信息服务 Web 开放接口类（Controller 层），负责接收前端 HTTP 请求并返回 JSON 响应。
CourseService	课程信息业务逻辑服务接口，定义课程模块对外暴露的公共业务操作方法。
CourseServiceImpl	课程信息业务逻辑服务实现类，实现 CourseService 接口，封装课程查询等核心逻辑。
CourseRepository	课程信息数据访问接口（Repository / DAO 层），继承 JpaRepository，由 Spring Data JPA 自动生成数据库 CRUD 实现。
Course	课程信息实体类，用于在各层之间传递数据，对应数据库中的课程表。

编写说明：

功能类表的作用是**提前告知读者**：Web 后端为了实现该功能，需要引入哪些核心类、每个类承担什么职责。这张表相当于后文类图和顺序图的“索引”，让读者在查看静态结构与动态交互之前，先对各参与类的分工有初步了解。学生填写时应注意“类名”与后文代码及类图严格保持一致，“描述”用一句话概括该类在本功能中的核心职责即可，避免写成通用的技术说明。

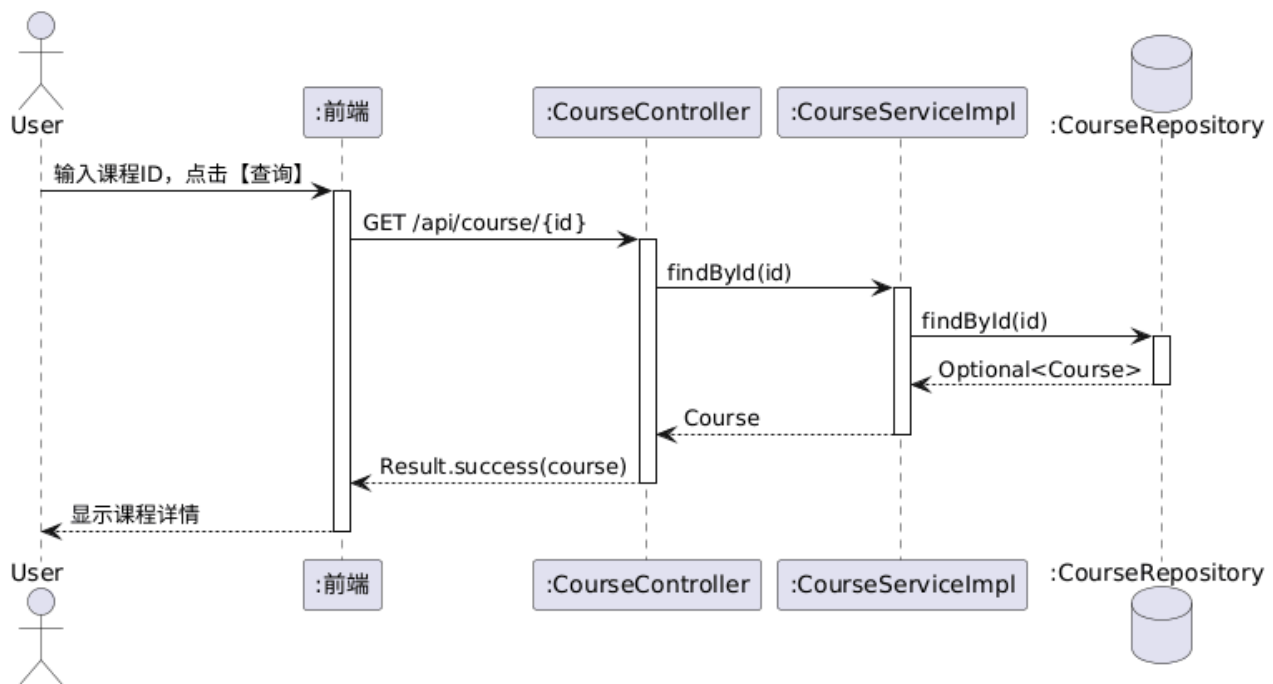
5、图 4-13 Web后端课程信息查询功能类类图



编写说明:

类图的作用是站在**静态结构视角**，展示实现该功能所需的类、接口及其相互关系（依赖、实现、继承）。读者通过类图可以快速把握系统的分层架构：Controller 依赖 Service 接口、Service 实现类实现 Service 接口并依赖 Repository、Repository 继承框架提供的 JpaRepository。学生在绘制时应遵循“只列类名和 public 方法”的原则，隐藏内部私有属性与复杂方法，突出类之间的关系线条，确保图面简洁、关系清晰。

6、图 4-14 Web后端课程信息查询流程的设计



编写说明：

顺序图（时序图）的作用是站在**动态运行视角**，按时间顺序展示对象之间的消息交互。与类图的“静态结构”互补，顺序图回答的是“代码跑起来时，谁先调用谁、传什么参数、返回什么结果”。学生在绘制时必须保留**生命线（Lifeline）**，并使用实线箭头表示同步调用、虚线箭头表示返回结果；参与者的排列顺序建议与代码分层一致（前端 → Controller → Service → Repository），使阅读方向自然流畅。

7、Web后端课程信息查询流程的正文说明

Web 后端课程信息查询流程说明如下：

（1）用户在前端页面输入需要查询的课程ID，点击【查询】按钮，前端将课程ID作为URL路径参数，通过 GET 请求发送至 Web 后端的 `CourseController`。

（2）`CourseController` 的 `findById()` 方法接收请求，利用 Spring MVC 的 `@PathVariable` 注解从 URL 路径中提取课程 ID 参数，随后调用 `CourseService` 接口的 `findById()` 方法进入业务逻辑层。

（3）`CourseServiceImpl` 实现类执行具体的业务逻辑：调用 `CourseRepository` 接口的 `findById()` 方法，由 Spring Data JPA 自动生成的数据访问实现去数据库中查询对应ID的课程

记录。查询结果以 `Optional<Course>` 形式返回给服务层，服务层取出其中的课程对象后返回给 Controller。

(4) `CourseController` 使用统一的 `Result` 工具类封装查询结果，以 JSON 格式回传给前端，前端解析响应数据后向用户展示课程的详细信息（如课程名称、学分、授课教师等）。

编写说明：

Web 后端流程正文说明的目的是对顺序图中的每一次消息交互进行细化解读，把“对象之间传了什么消息”翻译成“业务上做了什么处理、调用了什么方法、数据状态发生了什么变化”。这部分是连接“设计图”与“代码实现”的关键桥梁：读者对照顺序图和这段文字，就能明白代码中每一行对应的设计意图。学生在编写时应逐编号对应顺序图中的消息序号，使用“调用…方法”、“返回…对象”、“更新…属性”等精确动词，避免空泛的“系统进行处理”之类描述。

8、完整的课程信息查询功能设计

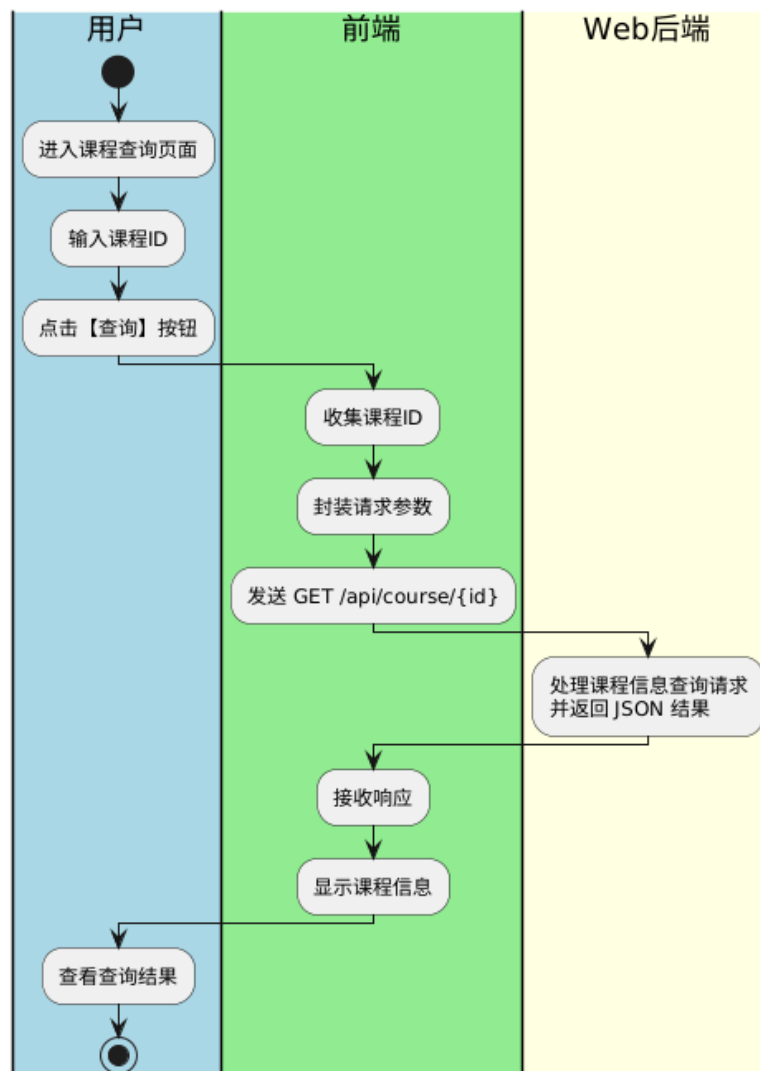
(1) 功能概述

课程信息查询功能是本系统的核心功能之一，旨在为用户提供便捷的课程信息检索能力。用户通过前端页面输入课程ID，系统调用Web后端接口查询数据库，将匹配的课程信息（如课程名称、学分、授课教师等）返回并展示给用户。本节采用自顶向下的设计描述方式，先从整体上描述该功能在用户、前端与Web后端之间的协作流程，再逐步细化到Web后端内部的类结构与处理流程，以建立从宏观到微观的完整认识。

(2) 整体协作过程设计

在描述具体实现之前，首先通过泳道图展示课程信息查询功能在用户、前端与Web后端三个参与方之间的整体协作过程，以便对该功能的执行脉络形成宏观把握。

图1 课程信息查询过程



课程信息查询过程说明如下：

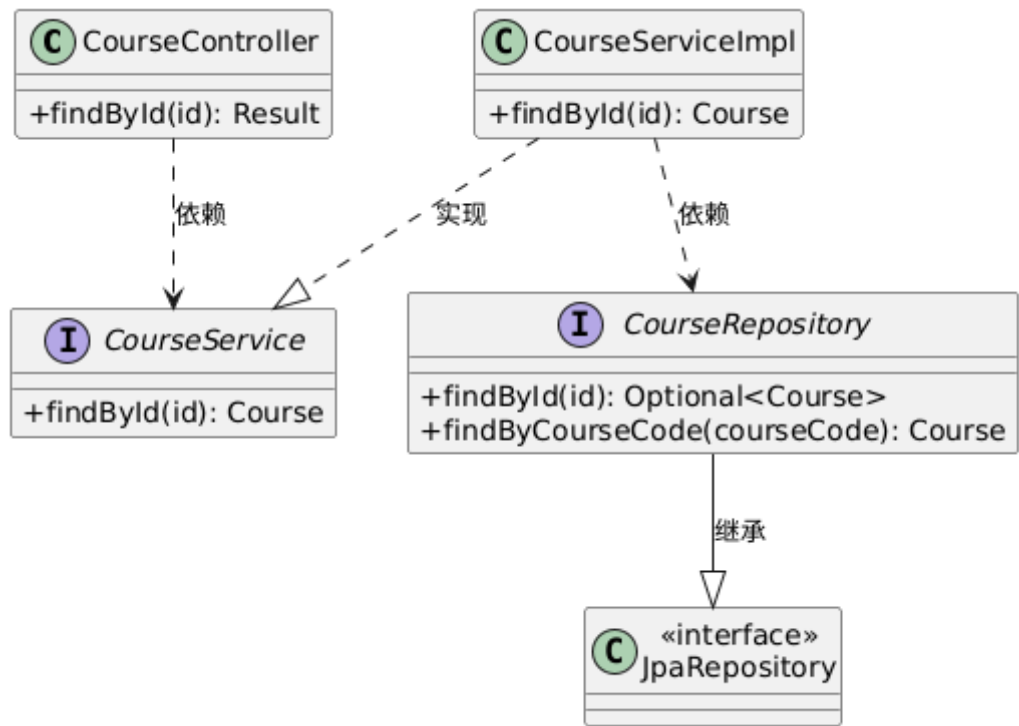
- ① 用户进入前端课程查询页面，输入需要查询的课程ID，点击【查询】按钮。
- ② 前端收集用户输入的课程ID，将其作为URL路径参数，通过 HTTP GET 请求发送至 Web 后端的 `/api/course/{id}` 接口。
- ③ Web 后端接收请求后，进行课程信息查询处理，并将结果以 JSON 格式返回给前端。Web 后端内部的具体处理流程将在后文图3中详细描述。前端接收响应后解析数据，向用户展示课程详细信息。

(3) Web后端详细设计

通过上述泳道图，已经了解了课程信息查询功能的整体协作流程。下面将进一步深入到Web后端内部，介绍实现该功能所需的核心类及其相互关系，并通过顺序图详细展示后端各层之间的调用过程。

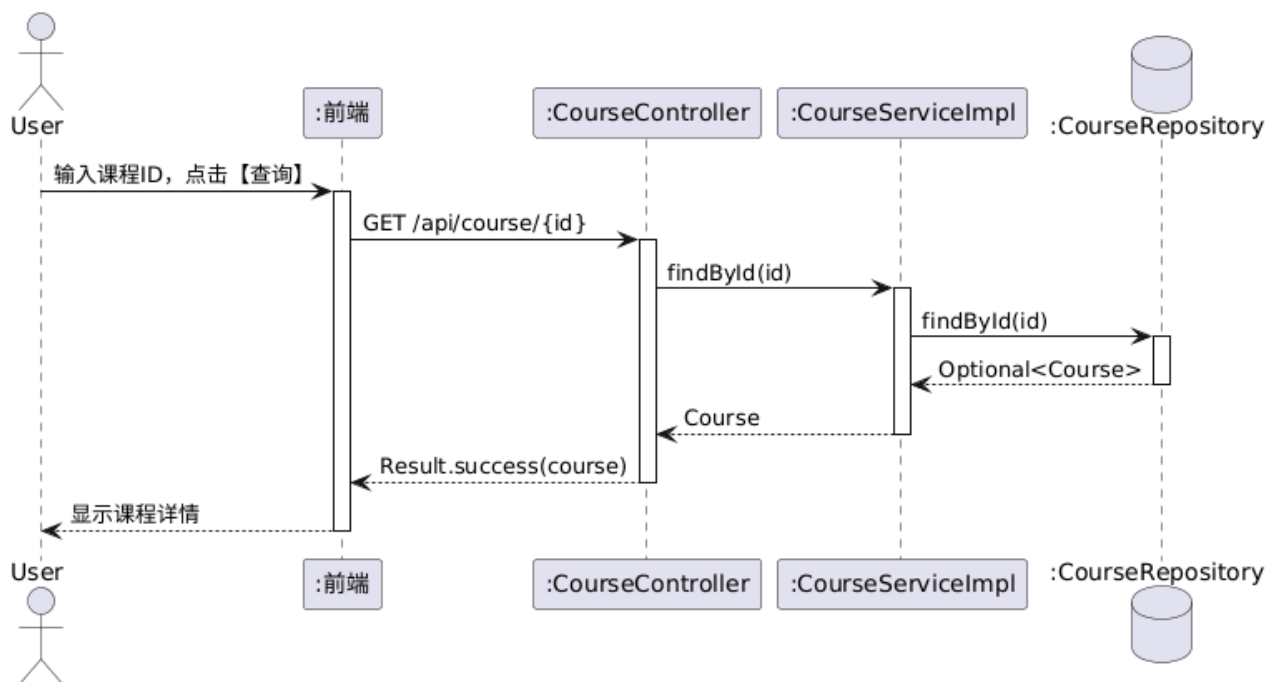
类名	描述
CourseController	课程信息服务 Web 开放接口类（Controller 层），负责接收前端 HTTP 请求并返回 JSON 响应。
CourseService	课程信息业务逻辑服务接口，定义课程模块对外暴露的公共业务操作方法。
CourseServiceImpl	课程信息业务逻辑服务实现类，实现 CourseService 接口，封装课程查询等核心逻辑。
CourseRepository	课程信息数据访问接口（Repository / DAO 层），继承 JpaRepository，由 Spring Data JPA 自动生成数据库 CRUD 实现。
Course	课程信息实体类，用于在各层之间传递数据，对应数据库中的课程表。

图2 Web后端课程信息查询功能类类图



类图从静态结构视角展示了实现该功能所需的类、接口及其相互关系（依赖、实现、继承）。通过类图可以快速把握系统的分层架构：Controller 依赖 Service 接口、Service 实现类实现 Service 接口并依赖 Repository、Repository 继承框架提供的 JpaRepository。上述类图与表1

图3 Web后端课程信息查询流程的设计



顺序图从动态运行视角，按时间顺序展示对象之间的消息交互。与类图的"静态结构"互补，顺序图呈现的是代码运行过程中各对象的调用时序：前端发送请求至 Controller，Controller 调用 Service，Service 调用 Repository 访问数据库，查询结果逐层返回，最终由 Controller 封装为统一响应回传前端。参与者的排列顺序与代码分层一致（前端 → Controller → Service → Repository），使阅读方向自然流畅。

Web 后端课程信息查询流程说明如下：

- ① 用户在前端页面输入需要查询的课程ID，点击【查询】按钮，前端将课程ID作为URL路径参数，通过 GET 请求发送至 Web 后端的 `CourseController`。
- ② `CourseController` 的 `findById()` 方法接收请求，利用 Spring MVC 的 `@PathVariable` 注解从URL路径中提取课程ID参数，随后调用 `CourseService` 接口的 `findById()` 方法进入业务逻辑层。
- ③ `CourseServiceImpl` 实现类执行具体的业务逻辑：调用 `CourseRepository` 接口的 `findById()` 方法，由 Spring Data JPA 自动生成的数据访问实现去数据库中查询对应ID的课程记录。查询结果以 `Optional<Course>` 形式返回给服务层，服务层取出其中的课程对象后返回给 Controller。
- ④ `CourseController` 使用统一的 `Result` 工具类封装查询结果，以 JSON 格式回传给前端，前端解析响应数据后向用户展示课程的详细信息（如课程名称、学分、授课教师等）。

9、功能设计编写的整体思路与要点

以上第8节给出了课程信息查询功能的完整设计示例。基于该示例，可以总结出功能设计编写存在以下几种常见描述思路，可根据系统的实际情况选择其中一种：

思路一：顶层整体描述 + 后端内部详细设计。 在顶层通过泳道图描述用户、前端与Web后端之间的整体协作过程，侧重展示用户与前端的交互细节；然后在底层详细展开Web后端内部的类结构（功能类表、类图）和调用流程（顺序图），重点阐述后端各层之间的协作机制。本文即采用此种思路，适用于以Web后端实现逻辑为核心关注点的功能模块。

思路二：顶层整体描述 + 前端内部详细设计。 在顶层同样通过泳道图描述整体协作过程，但侧重展示用户与前端、前端与后端的交互概貌；然后在底层详细展开前端内部的组件结构、状态流转或页面渲染流程，省略后端内部的类图和顺序图。此思路适用于前端交互逻辑较为复杂、需要重点呈现界面设计与用户体验的功能模块。

思路三：顶层整体描述 + 前后端内部都详细设计。 在顶层通过泳道图描述整体协作过程，然后在底层分别给出前端内部的详细设计（如组件结构图、页面跳转流程图等）和后端内部的详细设计（功能类表、类图、顺序图等）。此思路适用于前后端均有较复杂处理逻辑、且都需要在设计中予以充分说明的功能模块。

思路四：直接给出后端（或前端）详细设计。 若该功能模块不存在明显的前后端交互（如纯后台批处理任务、定时调度任务、命令行工具等），则无需绘制包含前端泳道的整体协作图，可直接从功能类表、类图和顺序图入手，描述系统内部的类结构与调用流程。此思路适用于无用户界面、无前后端交互的后台功能模块。

在具体编写时，无论采用上述哪种思路，均应注意以下要点：

① **自顶向下、逐层细化。** 功能设计应遵循自顶向下的描述思路，先从宏观层面把握整体协作过程，理解“谁参与了、做了什么、数据如何流转”；然后再逐步深入到系统内部，通过功能类表、类图和顺序图等手段，细化到具体的类职责、方法调用和参数传递。这种分层递进的描述方式符合认知规律，有助于建立起从全局到细节的完整理解。

② **图与文相互配合、互为补充。** 每种设计图都有其独特的表达侧重点——泳道图突出多参与方的协作过程，类图突出静态结构关系，顺序图突出动态交互时序。正文说明则对图中的每一环节进行文字级展开，把“图中发生了什么”翻译成“系统具体做了什么、数据如何传递”。图与文应逐条对应、紧密配合，既能直观把握整体，又能精确理解细节。

③ **前后衔接、形成闭环。** 整体协作过程的设计应为内部详细设计预留接口、埋下伏笔；内部详细设计则应回应前文提出的功能需求，展示具体的实现方案。例如，泳道图中Web后端用一个

处理框概括，而后续顺序图则展开其内部调用链，形成"先整体后细节"的完整叙事。各部分之间应注意衔接语句的使用，使全文逻辑连贯、过渡自然，避免产生"断层"之感。